

PromptIR : Prompting for All-in-One Blind Image Restoration

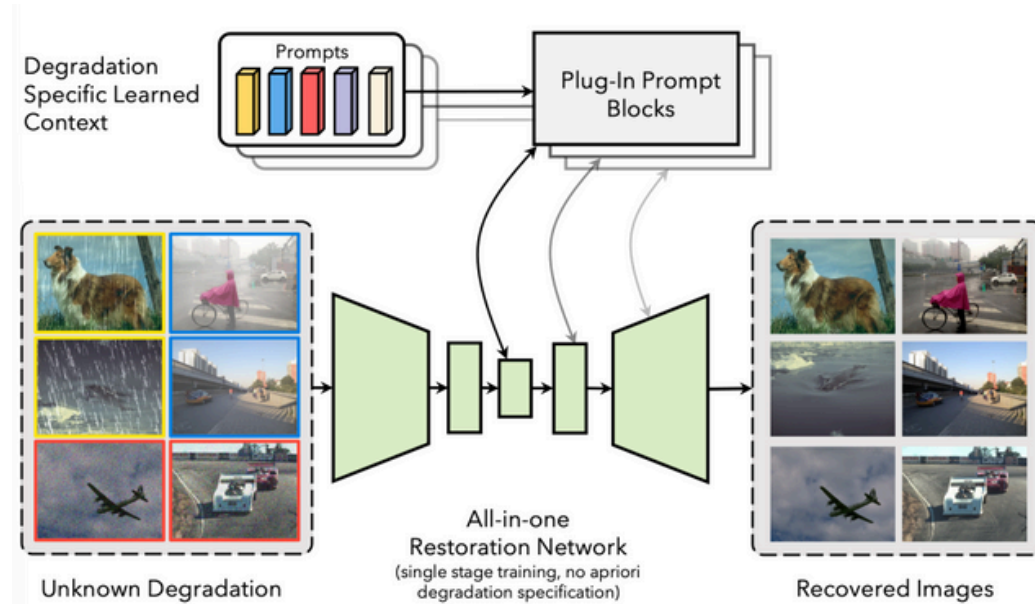
2211606 김소영

1 Introduction

- Image Restoration
 - Process of recovering a high-quality clean image from its degraded version
- Approach to address the Image Restoration problem
 - 1. Task-Specific**
 - Limitation
 - lack generalization beyond the specific degradation type and level
 - 2. Robust architecture + Implicit image prior**
 - Train separate copies of the same network on different datasets (degradation type, level, etc.)
 - Limitation
 - Replicating the same restoration model → compute-intensive, impractical
 - Requires degradation prior at test time
- **All-in-One IR**: 여러 degradation 유형과 강도에 모두 대응 가능한 IR 기법
 - 기존 연구: **AirNet**
 - employ '**Contrastive Learning**' paradigm
 - 추가 인코더를 두어, contrastive learning으로 학습시켜 서로 다른 degradation 유형을 구분하도록 학습
 - Limitation
 - 서로 다른 degradation 유형을 완전히 분리된 표현으로 학습하기 어려움
 - Additional encoder ⊕ Two-stage Training Approach → Training Burden ↑

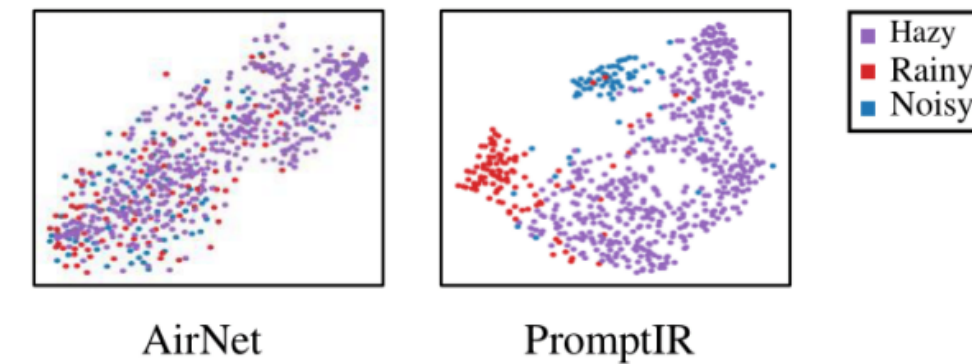
1 Introduction

- Proposed Solution: **Prompt-Learning-Based Approach**
 - **Prompt** = Degradation에 대한 중요한 discriminative information을 담은 **trainable parameter set**
 - **Interaction between Prompt and Feature Representaion**
 - With degradation-specific knowledge, **dynamically** enhance the representations
 - Dynamically adjusting behavior → Effective image restoration



Plug-and-play Prompt Module

- Degradation에 대해 명시적 분류 X
- implicit하게 degradation-conditioned prompt 생성
- 여러 decoding stage에 걸쳐 주입
- 이러한 구조가 all-in-one로, 다양한 restoration task 처리 가능



- Degradation Embedding Visualization (color = degradation type)
- Better clustering
- Prompt가 복원에 유용한 discriminative degradation context를 효과적으로 학습

1 Introduction

- **PromptIR의 Highlight**

- 1. Blind Restoration**

- Uses only the input image
 - No prior knowledge of degradation type or level required

- 2. Prompt Block = “Plug-in” module**

- Easily integrated into existing backbones
 - 구성
 - PGM(Prompt Generation Module)
 - generate input-conditioned prompts
 - PIM(Prompt Interaction Module)
 - use contextual information to guide the restoration network → effectively remove the corruption

- 3. Dynamic adaptation behavior** by achieving SOTA performance on various image restoration tasks

2 Related Work

Multi-degradation Image Restoration

1. Degradation-Specific Encoder/Decoder

- Train separate encoder/decoder pathways for each degradation type
- Limitation
 - Require prior knowledge of degradation type
 - Less scalable

2. Unified Model (Chen et al.)

- Shared backbone with task-specific heads/tails
- Require prior knowledge
- Not suitable for blind restoration
 - **Blind Restoration?**
 - Prior가 없는 상황에서 image restoration

3. Airnet (Li et al.)

- Image encoder trained through contrastive learning → good representation of the degradation → utilize representation for restoration
- Limitation
 - Two-stage training
 - Effectiveness of contrastive learning is sensitive to pair selection and data scale

4. Our Work

- **Single-stage** training for unified all-in-one image restoration task
- **Plug-and-play prompt module**, no additional encoder

2 Related Work

Transformer-based Image Restoration

- Transformer는 다양한 CV task에서 우수한 성능 보임
- strong feature representation capability로 이미지 복원 task에도 적용됨
- limitation of naive self-attention: quadratic complexity
- Effective Transformer Designs
 1. SwinIR
 - window-based self attention + convolution
 2. Restormer
 - multi-depth convolution head attention
- Our work
 - 높은 성능과 효율성을 가진 Restormer를 backbone으로 선택
 - + Generic Prompt Block

Prompt Learning

- Prompting-based method: provide in-context information to models to finetune on a target task
- **Learnable Prompt** (instead of fixed prompt): **parameter-efficient** adaption
- Our Goal
 - Learnable prompt와의 상호작용으로 입력에 따라 복원 동작을 동적으로 조절
 - Low-level image restoration을 위한 **generic model** 제안 (task-specific X)

3 Method

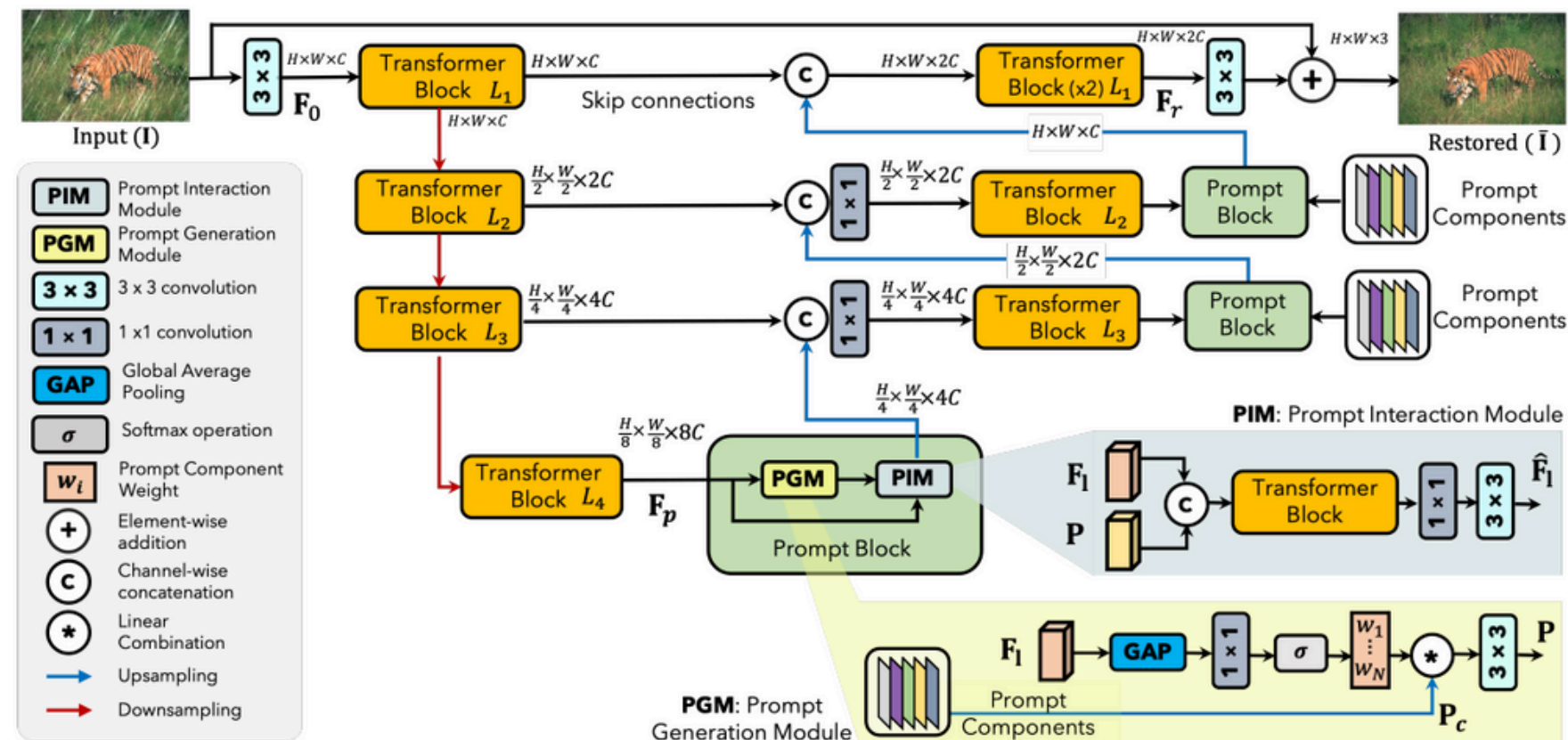
- “All-in-one” image restoration

- D(degradation)에 대한 사전 정보를 모르는 상태로, 이미지 복원하는 모델 학습
- 모델이 D의 특성에 아무것도 알지 못하는 상태에서는, D 유형의 간접적인 contextual 정보를 제공하여 성능 향상

- PromptIR

- 깨끗한 이미지를 복원할 때, D 유형과 관련된 정보를 모델에 보충하는 효율적인 모델
- **Prompt Block**: 학습가능한 prompt parameter 생성 → 복원 과정에서 해당 prompt를 이용해 모델 가이드

- Overall Pipeline



- encoding과 decoding 단계에 Transformer block을 포함한 UNet-style network
 - Transformer Block 결합 → channel-wise global context 처리 능력 향상
- Primary Component, **Prompt Block**
 1. **PGM** (Prompt Generation Module)
 - Input Feature + Prompt Component 을 사용해, input-conditioned prompt P 생성
 2. **PIM** (Prompt Interaction Module)
 - PGM에서 생성된 Prompt를 Transformer Block을 통해 input feature를 동적으로 조정
- Input으로부터 생성된 prompt가 decoder의 각 단계에서 상호작용 → 복원 과정에서 degradation-specific context를 강화시킴

3 Method

1. Input Image, $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$

2. Low-level Feature, $F_0 \in \mathbb{R}^{H \times W \times C}$
◦ 3×3 conv 통해 추출

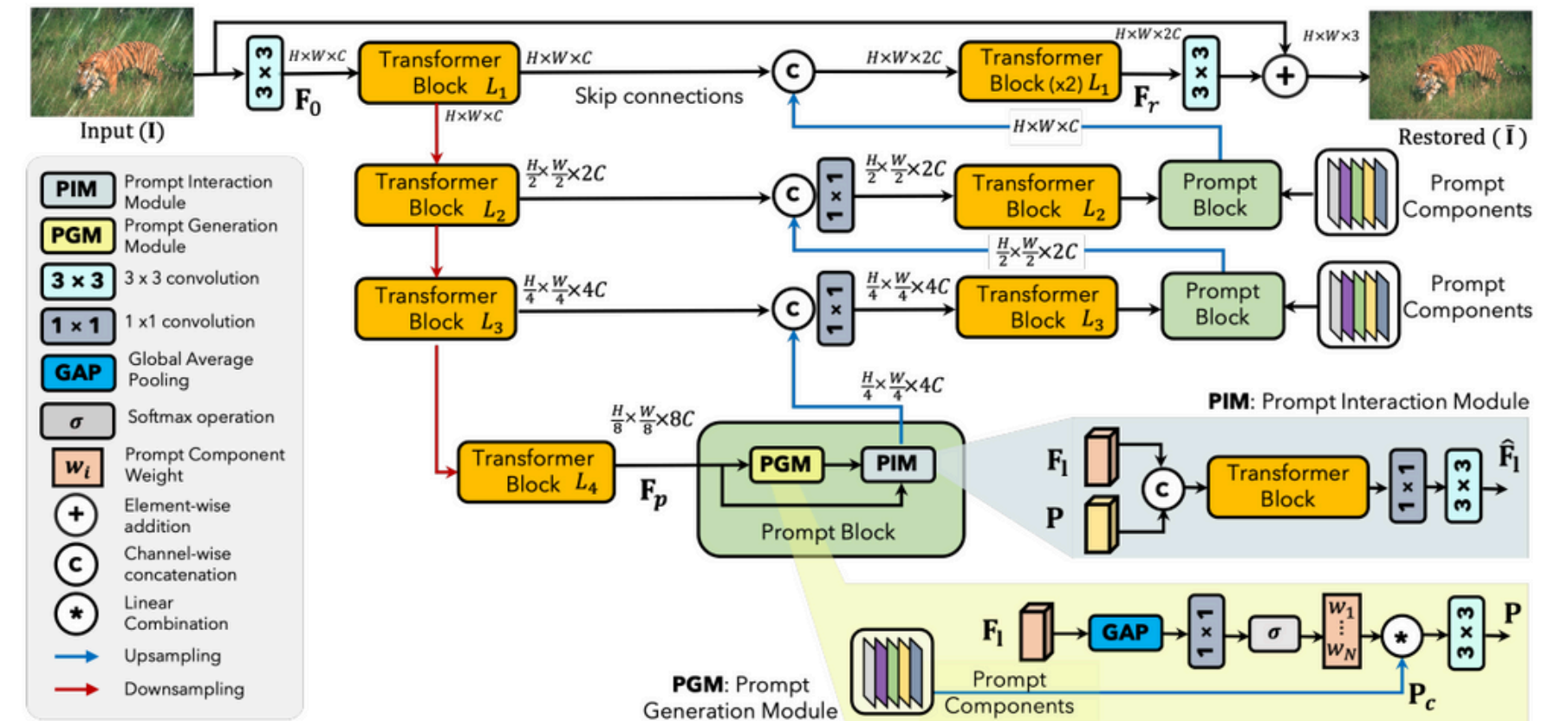
3. 4단계 계층적 Encoder-Decoder

3-1. Encoder

- Transformer Block $L_1 \rightarrow L_4$
- Encoder의 목적: 해상도 ↓ & 채널 수 ↑ $\Rightarrow$ “저해상도 잠재 표현(= F_l)”
- 문제: 더 깊어질수록,
 - 점진적으로 더 복잡한 패턴 학습 (채널 간 관계가 더 복잡)
 - 비선형 변환이 누적되어 더 복잡한 관계 모델링
- 위→아래로 가면서, Transformer Block의 개수 ↑
 - Transformer Block의 역할: Channel-wise interaction 강화

3-2. Decoder

- Decoder의 목적: 저해상도 → 고해상도의 깨끗한 출력으로 복원
- 이때 “Prompt Block” 도입
 - Decoder의 각 level 사이에 삽입
 - Degradation 유형에 대한 정보를 담음



3.1 Prompt Block

- **Prompt-based Technique**

- 대규모 사전학습 모델을 task에 맞게 적은 파라미터로 적응시킴
- 성능을 결정하는 요인: “**task-specific context**”를 **효율적으로 인코딩**했는가

- **Prompt Component**

- 입력 특징과 상호작용하는 학습 가능한 파라미터
 - 상호작용의 목적: **degradation 유형 정보**를 내재화

- **Overall Process of Prompt Block**

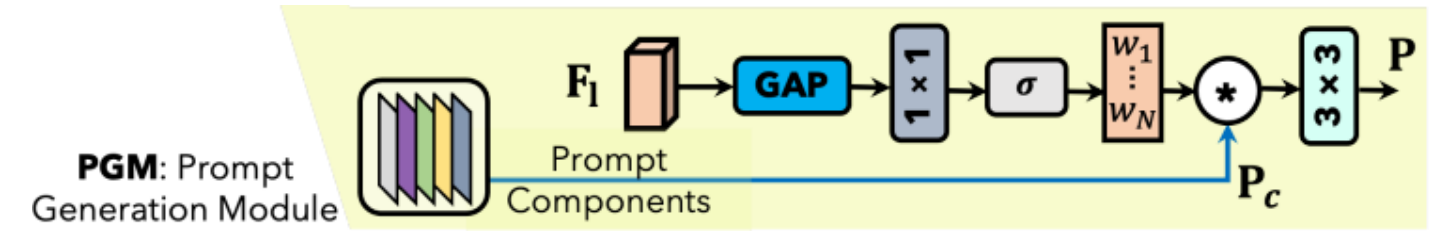
$$\hat{F}_l = \text{PIM}(\text{PGM}(P_c, F_l), F_l)$$

- Two Key Components: ① PGM ② PIM

3.1.1 Prompt Generation Module (PGM)

- Prompt Components (P_c)

- 학습가능한 파라미터의 집합
- 입력과 무관하게 고정되어 있음



- PGM

- 고정된 P_c 를 사용하면 입력별 차이를 반영 못함 → 성능 저하
- Input Feature에서 동적으로 가중치를 뽑아 P_c 와 결합 → Input-conditioned prompts P 생성
- 여러 Prompt Component들이 서로 지식을 공유할 수 있는(= 서로 연관된 복원 지식을 공유하는) shared space 형성 (← D들은 서로 완전히 독립적 X)

- PGM Process

$$\mathbf{P} = \text{Conv}_{3 \times 3} \left(\sum_{c=1}^N w_i \mathbf{P}_c \right), \quad w_i = \text{Softmax} (\text{Conv}_{1 \times 1} (\text{GAP}(\mathbf{F}_l)))$$

- 입력 특징 F_l = latent representation

- F_l : 여기서 채널은 여러 정보('edge', 'texture'...) 담고 있음

- GAP (Global Average Pooling)

- F_l 은 여러 채널로 구성됨 ($H \times W \times C$)
- GAP은 $H \times W \rightarrow 1 \times 1$ 로 변환시켜줌 (공간 차원을 평균내어 채널 차원만)
- 결과: $\mathbf{v} \in \mathbb{R}^{\hat{C}}$ (=채널별 스칼라 요약)

- $\text{Conv}_{1 \times 1}$ + Softmax

- 각 Prompt Component에 대응되는 score가 나옴
- softmax로 정규화하여, 가중치로 사용

- $\text{Conv}_{3 \times 3}$

- local spatial context 보완(=주변 위치 정보 반영)

- 문제: 입력이미지는 해상도가 다양하지만, P_c 는 고정 크기 → 해결방안: Bilinear Upsampling으로 P_c 의 spatial resolution을 F_l 크기에 맞춰 resize

3.1.2 Prompt Interaction Module (PIM)

- Prompt Block : plug-in module(=어떤 네트워크에나 붙일 수 있음)

$$\hat{F}_l = \text{Conv}_{3 \times 3}(\text{GDFN}(\text{MDTA}[F_l; P]))$$

1. Concat

- 입력특징 F_l - 프롬프트 P 를 “채널 차원”에서 연결
- 같은 채널공간에서 상호작용

2. Transformer

- Concatenated Representation $[F_l, P]$ 를 Transformer Block 내에 넣어줌
- **MDTA** (Multi-Dconv head Transposed Attention)

$$Y = W_p V \cdot \text{Softmax}\left(\frac{K \cdot Q}{\alpha}\right) + X$$

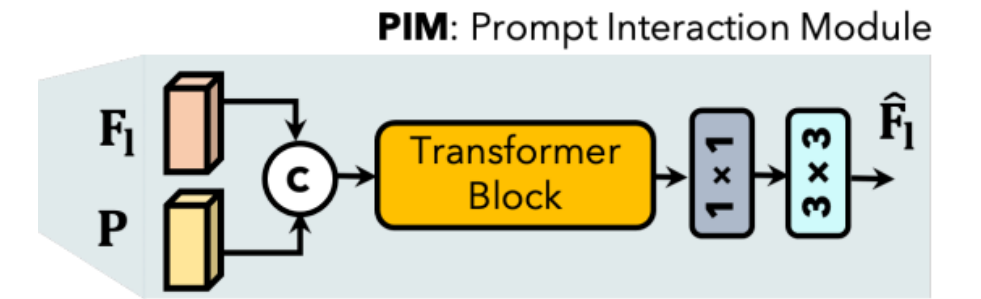
- F_l 은 degradation 정보가 부족함
- Self-Attention을 채널 차원에서 수행
- P 의 **degradation** 정보를 F_l 의 각 채널에 주입

⇒ **Degradation-specific Representation**

- **GDFN** (Gated-Dconv feedforward network)

$$Z = W_p^0 \phi(W_d^1 W_p^1 (\text{LN}(Y))) \odot W_d^2 W_p^2 (\text{LN}(Y)) + Y$$

- **중요도에 따라 gating**
- P 정보가 반영된 feature 중 중요한 특성은 강화하고, 덜 중요한 특성은 억제
- 특징을 선택적으로 변환



3.1 Prompt Block

정리해보자면,

- **PGM**
 - 과정
 - 입력 특징 F_l 에서 동적으로 가중치 w 를 뽑아, Prompt Component P_c 가중합
 - **input-conditioned prompt P 생성**
 - 목적
 - **입력 이미지의 degradation 상태를 반영한 PROMPT**
- **PIM**
 - 과정
 - F_l 과 PGM을 통해 생성된 프롬프트 P 를 채널 차원에서 concat
 - Transformer block을 통해 상호작용 → 새로운 특징 $\hat{F}_l$ 산출
 - 목적
 - **Prompt 정보를 실제 특징 변환에 주입 → Degradation 복원에 특화된 표현 학습**
 - 결과는 다음 복원 연산의 입력으로 들어감

4 Experiments

- 3 image restoration tasks: ① Image dehazing ② Image Deraining ③ Image Denoising
- 2 Experimental Settings
 - a. All-in-One Setting
 - Unified model that can recover images across all degradation types
 - b. Single-task Setting
 - Separate models for different restoration tasks
- Implementation Details
 - end-to-end trainable
 - no pretraining for any individual component
 - 4-level encoder-decoder, with varying numbers of Transformer blocks at each level [4,6,6,8]
 - 연속된 2개의 decoder level 사이마다 하나의 prompt block 배치 → 3개의 prompt block
 - Prompt Component의 개수: 5개
- Training Setting
 - Batch size: 32(all-in-one), 8(single-task)
 - L1 loss
 - Adam Optimizer ($\beta_1 = 0.9, \beta_2 = 0.999$)
 - Learning Rate: 2×10^{-4}
 - 200 Epochs
- Input & Augmentation
 - Training input: 128×128 cropped patches
 - Data augmentation: random horizontal & vertical flip

4 Experiments

- **Datasets**

- Image Denoising (Single-task)
 - Training datasets: BSD400, WED
 - Noise generation: Gaussian noise with $\sigma \in \{15, 25, 50\}$
 - Testing datasets: BSD68, Urban100
- Image Deraining (Single-task)
 - Dataset: Rain100L
- Image Dehazing (Single-task)
 - Dataset: SOTS
- All-in-One Setting
 - Combine all four Combined training using all datasets from single-task settings
 - Single unified model trained for all restoration tasks

4.1 Multiple Degradation All-in-One Results

“하나의 모델로 여러 task를 처리할 수 있는가”

Table 1: Comparisons under All-in-one restoration setting: single model trained on a combined set of images originating from different degradation types. When averaged across different tasks, our PromptIR provides a significant gain of 0.86 dB over the previous all-in-one method AirNet [29].

Method	Dehazing on SOTS [31]	Deraining on Rain100L [16]	Denoising on BSD68 dataset [41])			Average
			$\sigma = 15$	$\sigma = 25$	$\sigma = 50$	
BRDNet [51]	23.23/0.895	27.42/0.895	32.26/0.898	29.76/0.836	26.34/0.836	27.80/0.843
LPNet [17]	20.84/0.828	24.88/0.784	26.47/0.7782	24.77/0.748	21.26/0.552	23.64/0.738
FDGAN [14]	24.71/0.924	29.89/0.933	30.25/0.910	28.81/0.868	26.43/0.776	28.02/0.883
MPRNet [70]	25.28/0.954	33.57/0.954	33.54/0.927	30.89/0.880	27.56/0.779	30.17/0.899
DL[16]	26.92/0.391	32.62/0.931	33.05/0.914	30.41/0.861	26.90/0.740	29.98/0.875
AirNet [29]	27.94/0.962	34.90/0.967	33.92/0.933	31.26/0.888	28.00/0.797	31.20/0.910
PromptIR (Ours)	30.58/0.974	36.37/0.972	33.98/0.933	31.31/0.888	28.06/0.799	32.06/0.913

- **Average**
 - +0.86 dB over AirNet
 - +2.08 dB over DL
 - **Dehazing**
 - +2.64 dB PSNR
 - Largest performance gain
- **Deraining**
 - +1.47 dB over AirNet
 - +3.73 dB over DL
 - **Denoising**
 - +1.16 dB over DL



Figure 4: **Dehazing comparisons** for all-in-one methods on images from the SOTS dataset [31]. The image quality of the results produced by our PromptIR is visually better than the previous state-of-the-art approach AirNet[29].



Figure 5: **Image deraining comparisons** for all-in-one methods on images from the Rain100L dataset [16]. Our method effectively removes rain streaks to generate rain-free images.



Figure 6: **Denoising results** for all-in-one methods.

4.2 Single Degradation One-by-One Results

“Prompt block이 single task인 경우에도 도움이 되는가”

Table 2: Dehazing results in the single-task setting on the SOTS benchmark dataset [31]. Our PromptIR achieves a significant boost of 8.13 dB over AirNet [29].

Method	DehazeNet[4]	MSCNN[46]	AODNet[30]	EPDN[44]	FDGAN[14]	AirNet[29]	Restormer[67]	PromptIR (Ours)
PSNR	22.46	22.06	20.29	22.57	23.15	23.18	<u>30.87</u>	31.31
SSIM	0.851	0.908	0.877	0.863	0.921	0.900	<u>0.969</u>	0.973

Table 3: Deraining results in the single-task setting on Rain100L [16]. Compared to the AirNet [29] algorithm, the proposed method yields 2.13 dB PSNR improvement.

Method	DIDMDN[71]	UMR[65]	SIRR[62]	MSPFN[24]	LPNet[17]	AirNet[29]	Restormer[67]	PromptIR(Ours)
PSNR	23.79	32.39	32.37	33.50	33.61	34.90	<u>36.74</u>	37.04
SSIM	0.773	0.921	0.926	0.948	0.958	0.977	<u>0.978</u>	0.979

Table 4: Denoising comparisons in the single-task setting on BSD68 [41] and Urban100 [22] datasets. For the challenging noise level of $\sigma = 50$ on Urban100 [22], our PromptIR obtains 0.51 dB gain compared to AirNet [29].

Method	BSD68 [41]			Urban100 [22]		
	$\sigma = 15$	$\sigma = 25$	$\sigma = 50$	$\sigma = 15$	$\sigma = 25$	$\sigma = 50$
CBM3D [11]	33.50/0.922	30.69/0.868	27.36/0.763	33.93/0.941	31.36/0.909	27.93/0.840
DnCNN [74]	33.89/0.930	31.23/0.883	27.92/0.789	32.98/0.931	30.81/0.902	27.59/0.833
IRCNN [75]	33.87/0.929	31.18/0.882	27.88/0.790	27.59/0.833	31.20/0.909	27.70/0.840
FFDNet [76]	33.87/0.929	31.21/0.882	27.96/0.789	33.83/0.942	31.40/0.912	28.05/0.848
BRDNet [51]	34.10/0.929	31.43/0.885	28.16/0.794	34.42/0.946	31.99/0.919	28.56/0.858
AirNet [29]	34.14/0.936	31.48/0.893	28.23/0.806	34.40/0.949	32.10/0.924	28.88/0.871
Restormer [67]	<u>34.29/0.937</u>	<u>31.64/0.895</u>	<u>28.41/0.810</u>	<u>34.67/0.969</u>	<u>32.41/0.927</u>	<u>29.31/0.878</u>
PromptIR(Ours)	34.34/0.938	31.71/0.897	28.49/0.813	34.77/0.952	32.49/0.929	29.39/0.881

- **Dehazing**
 - +8.13 dB over AirNet
 - +0.44 dB over Restormer
- **Deraining**
 - +2.13 dB over AirNet
- **Denoising ($\sigma=50$)**
 - +0.51 dB over AirNet

4.3 Ablations Studies

(A) Impact of PGM

Table 5: Impact of PGM. Results are reported on Rain100L [64] dataset.

Method	PSNR
Baseline [67]	36.74
Fixed-Prompt	36.85
Dynamic-Prompt	37.04

- PromptIR 네트워크에 prompt block을 추가함으로써 baseline 대비 0.3 dB의 성능 향상이 있음
- PGM을 통해 입력 내용에 input-conditioned dynamic prompt를 생성하는 것이 fixed prompt를 사용하는 경우보다 0.19 dB의 추가 성능 향상을 제공

(B) Position of Prompt Blocks

Table 6: Prompt blocks position. Results are reported on Rain100L [64] dataset.

Model	PSNR
level 4 (latent)	36.76
levels 4+3	36.84
levels 4+3+2	37.04

- PromptIR의 계층적 구조에서 decoder 쪽 어느 위치에 prompt block을 둘 것인지 실험
- latent space에 하나의 prompt block만 사용하는 경우 네트워크 성능이 오히려 저하됨
- decoder의 모든 인접한 레벨 사이에 prompt block을 배치하는 경우 가장 좋은 성능을 보

(C) Generalization to Unseen Degradation Level

Table 7: Denoising comparisons on unseen noise level of $\sigma = 100$.

	BSD68 [41]	Urban100 [22]
Method	PSNR	PSNR
Airnet [29]	13.64	13.72
PromptIR (Ours)	21.03	20.50

- unseen degradation level에 대한 일반화 성능 평가를 위해, 학습에 사용되지 않은 $\sigma = 100$ 노이즈 수준에서 테스트
- PromptIR이 AirNet과 비교했을 때 약 7 dB의 PSNR 차이 → 상대적으로 뛰어난 일반화 성능

4.3 Ablations Studies

(D) Performance on spatially variant degradation

Table 8: Evaluation on Spatially Variant Degradation on BSD68 [41] test set.

Model	PSNR
Airnet [29]	31.42
PromptIR (Ours)	31.65

- 이미지 내 위치마다 noise 강도가 다르게 설정한 degradation(공간적으로 불균일한 degradation)에도 적용 가능한가
- PromptIR가 Airnet 대비 0.23 dB의 성능향상 있음

(E) Training model with different combinations of degradation

Table 9: Performance of the proposed PromptIR framework, when trained on different combinations of degradation types (tasks) i.e., removal of noise, rain and haze.

Degradation			Denoising on BSD68 dataset [41]			Deraining on Rain100L [16]	Dehazing on SOTS [31]
Noise	Rain	Haze	$\sigma = 15$	$\sigma = 25$	$\sigma = 50$		
✓	✗	✗	34.34/0.938	31.71/0.898	28.49/0.813	-	-
✗	✓	✗	-	-	-	37.03/0.9786	-
✗	✗	✓	-	-	-	-	31.31/0.929
✓	✓	✗	34.26/0.937	31.61/0.895	28.37/0.810	39.32/0.986	-
✓	✗	✓	33.69/0.928	31.03/0.880	27.74/0.777	-	30.09/0.975
✗	✓	✓	-	-	-	35.12/0.969	30.36/0.973
✓	✓	✓	33.98/0.933	31.31/0.888	28.06/0.799	36.37/0.972	30.58/0.974

- Degradation task의 조합이 PromptIR의 성능에 미치는 영향 평가
- Degradation type의 수가 증가할수록 이미지 복원이 점점 더 어려워짐 → 성능 감소
- 특히 dehazing이 포함된 데이터셋일 경우, 모델 성능에 부정적 영향
- 흥미로운 점: rainy and noisy image를 함께 학습한 모델 → 좋은 성능 → deraining과 denoising 작업 간의 긍정적인 상관관계 (Airnet 연구에서도 동일하게 관찰)

5 Conclusion

Limitations of Existing Methods

- Task-specific restoration models
- Lack Generalization
- Require prior knowledge of degradation type

PromptIR

- propose “**Drop-in prompt block**” that can dynamically adapts feature representations
- Enables **blind all-in-one image restoration**
- **SoTA** restoration model that leads to significant improvements on image denoising, deraining, and dehazing tasks